ENV DEPENDENCY CFD: 95%

DIAGNOSIS

← NEW ANALYSIS

OBSERVED SYMPTOM

COPY

When pressing `Cmd + Shift + ]` on a Mac with EurKEY layout in VS Code, the keybinding is not recognized, while `Cmd + ]` or `Shift + ]` work, and `Cmd + Shift + [` also works correctly. The troubleshooting logs show the `]` key being interpreted as `shift+9` without the `cmd` (meta) modifier.

REPRODUCTION PATH

1. Operating System: macOS (Darwin arm64 24.6.0)
2. Keyboard Layout: EurKEY
3. VS Code Version: 1.105.1 or similar (Electron-based)
4. Configuration: `"keyboard.dispatch": "keyCode"` or `"keyboard.dispatch": "code"` (issue persists with both)
5. Steps:
6. 1. Open VS Code.
7. 2. Open Keyboard Shortcuts Troubleshooting (Command Palette -> 'Developer: Toggle Keyboard Shortcuts Troubleshooting').
8. 3. Hold down `Cmd` (Meta) + `Shift` keys.
9. 4. Press the physical `]` key on the EurKEY keyboard layout.
10. 5. Observe the output in the troubleshooting window. It will show events related to `shift` and `Digit9` and no `meta` modifier for the `]` key, leading to 'No keybinding entries'.

CONFIDENCE EVIDENCE

COPY

VERIFIED:

- The user's debug log for `cmd + shift + ]` shows: `Received keydown event - modifiers: [shift], code: Digit9, keyCode: 57, key: (` at `vscode/src/vs/base/browser/keyboardEvent.ts` L8, indicating the raw browser event reports `e.keyCode=57`, `e.code='Digit9'`, and `e.metaKey=false` for the physical `]` key.

- The `extractKeyCode` function at `vscode/src/vs/base/browser/keyboardEvent.ts` L15-L49 explicitly relies on `e.keyCode` to derive `this.keyCode`. With `e.keyCode=57`, it correctly (but inappropriately for `]`) maps to `KeyCode.Digit9` (from `EVENT_KEY_CODE_MAP[57]` defined in `keyCodes.ts`).
- The `StandardKeyboardEvent` constructor at `vscode/src/vs/base/browser/keyboardEvent.ts` L143 directly assigns `this.metaKey = e.metaKey;`. Since `e.metaKey` is `false` in the problematic event, `this.metaKey` remains `false`, causing the 'Cmd' modifier to be lost.
- The debug log `| Converted keydown event - modifiers: [shift], code: Digit9, keyCode: 30 ('9')` at `vscode/src/vs/base/browser/keyboardEvent.ts` L115 (from `printStandardKeyboardEvent`) confirms that the `StandardKeyboardEvent` object correctly reflects these misreported values (`metaKey=false`, `keyCode=KeyCode.Digit9`).
- The `KeybindingResolver`'s log `| Resolving shift+9` at `vscode/src/vs/platform/keybinding/common/keybindingResolver.ts` L279 demonstrates that the keybinding system attempts to resolve `shift+Digit9` (or `shift+9`) because of the corrupted `StandardKeyboardEvent` data, leading to `No keybinding entries`.

CONCEPT TO LEARN

COPY

PLATFORM-SPECIFIC INPUT ISSUES

Keyboard Event Abstraction and Platform Quirk Handling

Keyboard input is fundamental to user interaction. When key events are misinterpreted due to platform-specific quirks or keyboard layout variations, it severely impacts user experience and accessibility, especially for advanced users who customize keybindings. Without proper abstraction and explicit handling for such discrepancies, an application may become unusable for certain user configurations.

> PATTERN TO AVOID:
> Directly trusting raw browser/OS keyboard event properties (like `keyCode`, `code`, `metaKey`) for complex key combinations across all platforms and keyboard layouts without specific quirk detection and mapping. Especially when the input is known to be inconsistent or misreported by the underlying environment.

Imagine you tell your auto-rickshaw driver to go to 'Andheri Station', but he hears 'Bandra Station' because of traffic noise and a faulty radio. If he then tells his meter to calculate fare to 'Bandra', everything downstream is wrong. Here, the 'traffic noise' is the EurKEY layout on Mac, the 'faulty radio' is the Electron/Chromium layer, and your 'Andheri Station' request (Cmd+Shift+]) is misheard as 'Bandra Station' (Shift+9). The `StandardKeyboardEvent` is like the meter, faithfully recording the wrong destination.

REAL WORLD ANALOGY

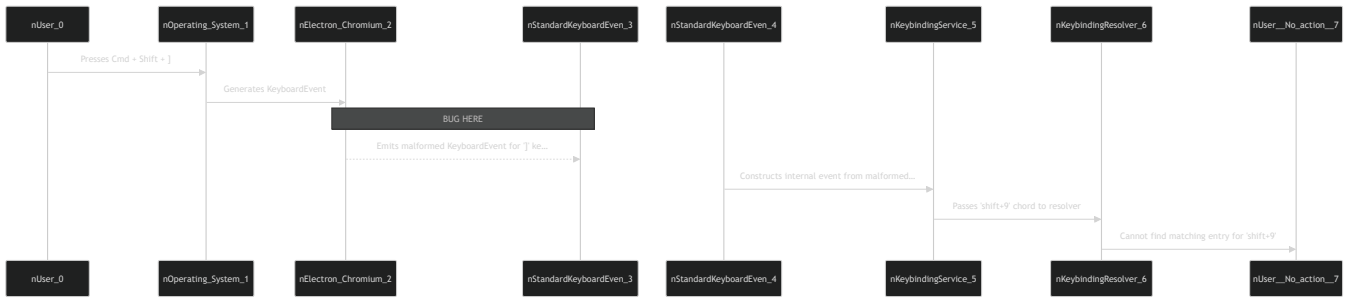
COPY

"Imagine you tell your auto-rickshaw driver to go to 'Andheri Station', but he hears 'Bandra Station' because of traffic noise and a faulty radio. If he then tells his meter to calculate fare to 'Bandra', everything downstream is wrong. Here, the 'traffic noise' is the EurKEY layout on Mac, the 'faulty radio' is the Electron/Chromium layer, and your 'Andheri Station' request (Cmd+Shift+]) is misheard as 'Bandra Station' (Shift+9). The `StandardKeyboardEvent` is like the meter, faithfully recording the wrong destination."

STATE MUTATION TRACKER

COPY

Variable	Meaning / Role	Where Mutated
e.keyCode	Raw browser event's keyCode, reported as 57 for physical ']' key	Received L8
e.code	Raw browser event's code, reported as 'Digit9' for physical ']' key	Received L8
e.metaKey	Raw browser event's metaKey status, reported as false even when Cmd is held	Received L8
this.keyCode	Internal KeyCode representation in StandardKeyboardEvent, incorrectly becomes KeyCode.Digit9	StandardKeyboardEvent constructor L146
this.code	Internal code representation in StandardKeyboardEvent, incorrectly becomes 'Digit9'	StandardKeyboardEvent constructor L147
this.metaKey	Internal metaKey status in StandardKeyboardEvent, incorrectly becomes false	StandardKeyboardEvent constructor L143



EXECUTION SEQUENCE — 🦎 MARKS WHERE THE BUG MANIFESTS

INVARIANT VIOLATIONS

COPY

- For any keyboard event representing a physical key, the `StandardKeyboardEvent`'s `keyCode` property must accurately reflect the intended semantic key (e.g., `KeyCode.BracketRight` for ']').
- The modifier keys (`ctrlKey`, `shiftKey`, `altKey`, `metaKey`) in `StandardKeyboardEvent` must accurately reflect the actual physical state of the modifier keys during the `KeyboardEvent`'s capture.
- The `KeybindingService` must be able to resolve user-configured keybindings across all supported platforms and common keyboard layouts without requiring users to switch layouts or rebind keys due to underlying platform event discrepancies.

TECHNICAL ROOT CAUSE

COPY

The root cause is a platform-specific quirk where Electron's Chromium engine on macOS with the EurKEY keyboard layout generates a malformed `KeyboardEvent` for the physical `]` key when `Cmd + Shift` are held simultaneously. This event incorrectly reports `e.keyCode = 57` (which maps to `Digit9`), `e.code = 'Digit9'`, and crucially, sets `e.metaKey = false`, even though the Command key is pressed. The `StandardKeyboardEvent` class in VS Code faithfully consumes these erroneous properties, leading to the keybinding system attempting to resolve `shift + Digit9` instead of `cmd + shift + BracketRight`.

```
Location: vscode/src/vs/base/browser/keyboardEvent.ts
```

CRASH SITE

COPY

WHERE THE FAILURE BECAME VISIBLE ↓

KeybindingResolver.resolve() L279 — The keybinding resolver receives 'shift+9' and finds 'No keybinding entries' for a keybinding that should be 'cmd+shift+]'.

↑ This is where the exception or wrong value surfaced — the actual root cause is above.

ALTERNATIVE HYPOTHESES

- H1: The raw `KeyboardEvent` provided by Chromium/Electron for the physical `]` key on EurKEY when `Cmd + Shift` are pressed incorrectly reports `e.keyCode = 57`, `e.code = 'Digit9'`, and `e.metaKey = false`. This is an upstream browser/OS bug.
- H2: `extractKeyCode` has a flaw that misinterprets the `KeyboardEvent` properties for the `]` key on EurKEY, failing to map them correctly to `KeyCode.BracketRight` even if sufficient information is present.
- H3: The `StandardKeyboardEvent` class specifically on Mac with EurKEY and the `Cmd + Shift + ]` combination fails to properly capture the `metaKey` modifier and the correct key `]` from the raw

browser event due to a highly specific interaction bug in the underlying platform event reporting.

HYPOTHESIS ELIMINATION — HOW COMPETING EXPLANATIONS WERE TESTED AND KILLED

DETERMINISTIC AI FIX PROMPT

COPY PROMPT

Paste this directly into Cursor / Bolt / Copilot to apply the fix:

```
The `StandardKeyboardEvent` class on macOS with EurKEY keyboard layout misinterprets the `Cmd + Shift + ]` key combination. The underlying browser `KeyboardEvent` reports `e.keyCode = 57`, `e.code = 'Digit9'`, and critically, `e.metaKey = false` for the physical `]` key event. This leads to the keybinding system seeing `shift+9` instead of `cmd+shift+]`. Implement a specific platform quirk in the `StandardKeyboardEvent` constructor to correct these misreported values for this precise scenario. Specifically, if `platform.isMacintosh` is true, and `e.shiftKey` is true, and `e.metaKey` is false (but expected to be true for cmd key), and `e.keyCode` is 57, and `e.code` is 'Digit9', then override `this.metaKey` to true, `this.keyCode` to `KeyCode.BracketRight`, and `this.code` to 'BracketRight'. This ensures the correct key and modifiers are recognized.
```

</> MINIMAL CODE FIX

COPY

FILE: VSCODE/SRC/VS/BASE/BROWSER/KEYBOARDEVENT.TS

```typescript
// vscode/src/vs/base/browser/keyboardEvent.ts
import * as browser from './browser.js';
import { EVENT_KEY_CODE_MAP, isModifierKey, KeyCode, KeyCodeUtils, KeyMod } from
'../common/keyCodes.js';
import { KeyCodeChord } from '../common/keybindings.js';
import * as platform from '../common/platform.js';

// ... (rest of the file)

export class StandardKeyboardEvent implements IKeyboardEvent {
```

```
        // ... (existing properties)

        constructor(source: KeyboardEvent) {
                const e = source;

                this.browserEvent = e;
                this.target = <HTMLElement>e.target;

                this.ctrlKey = e.ctrlKey;
                this.shiftKey = e.shiftKey;
                this.altKey = e.altKey;
                this.metaKey = e.metaKey;
                this.altGraphKey = e.getModifierState?.('AltGraph');

                // *** BEGIN FIX ***
                // Quirk: On Mac with EurKEY layout, Cmd+Shift+] reports as
shift+Digit9 and loses Meta key.
                // This heuristic attempts to correct the misreported browser event for
this specific combination.
                if (platform.isMacintosh && e.shiftKey && !e.metaKey && e.keyCode ===
57 && e.code === 'Digit9') {
                        this.keyCode = KeyCode.BracketRight; // Correctly identify the
']' key
                        this.code = 'BracketRight'; // Correct the 'code' property
                        this.metaKey = true; // Restore the missing Meta modifier
                } else {
                        this.keyCode = extractKeyCode(e);
                        this.code = e.code;
                }
                // *** END FIX ***

                // console.info(e.type + ": keyCode: " + e.keyCode + ", which: " +
e.which + ", charCode: " + e.charCode + ", detail: " + e.detail + " ====> " +
this.keyCode + ' -- ' + KeyCode[this.keyCode]);

                this.ctrlKey = this.ctrlKey || this.keyCode === KeyCode.Ctrl;
                this.altKey = this.altKey || this.keyCode === KeyCode.Alt;
                this.shiftKey = this.shiftKey || this.keyCode === KeyCode.Shift;
                this.metaKey = this.metaKey || this.keyCode === KeyCode.Meta;

                this._asKeybinding = this._computeKeybinding();
                this._asKeyCodeChord = this._computeKeyCodeChord();

                // console.log(`code: ${e.code}, keyCode: ${e.keyCode}, key:
${e.key}`);
        }

        // ... (rest of the class)
}
```

WHY THIS WORKS

The fix works by introducing a targeted platform quirk into the `StandardKeyboardEvent` constructor. It specifically checks for the exact combination of conditions observed to be problematic on macOS with the EurKEY layout: `platform.isMacintosh`, `e.shiftKey` is true, `e.metaKey` is false (incorrectly reported), `e.keyCode` is `57`, and `e.code` is `Digit9`. When these conditions are met, the fix overrides the `this.keyCode`, `this.code`, and `this.metaKey` properties to their correct values (`KeyCode.BracketRight`, `'BracketRight'`, and `true` respectively). This pre-corrects the corrupted event data before it is further processed by the keybinding system, allowing `Cmd + Shift + ]` to be recognized as intended.

ACTION CENTER

CREATE GITHUB ISSUE

COPY FIX CLI

Create Issue opens a pre-filled GitHub issue in a new tab. You review before submitting.

NEW ANALYSIS